Colônia de Formigas Distribuída para o Caixeiro-Viajante com Eleição de Líder, Relógio de Lamport e Exclusão Mútua

Pedro Henrique Vilaça Valverde[1], **Marcus Prado Silva**[1], **Gabriel Batista de Almeida**[1], **Eduardo Alves Salgado Lisboa Oliveira**[1]

[1]Instituto de Ciências Exatas e Informática (ICEx) – PUC Minas
Belo Horizonte – MG – Brasil

Resumo. *Este trabalho apresenta uma colônia de formigas distribuída para o problema do caixeiro-viajante, executada por nós Java simétricos que trocam mensagens por gRPC. A matriz de feromônio é replicada e somente pode ser alterada dentro de uma seção crítica, cuja coordenação combina eleição de líder pelo algoritmo Bully, detecção de falha por heartbeat, relógio lógico de Lamport e exclusão mútua de Ricart-Agrawala. Em ensaios com quatro nós e a instância berlin52, as réplicas convergiram para matrizes idênticas e o cluster atingiu a rota de 7542, que é o ótimo conhecido. O failover do líder levou cerca de 10 s no perfil de rede local, e a primeira rodada após a queda caiu de 15,6 s para 0,56 s depois de corrigirmos o broadcast. A limitação central — não distinguir nó lento de nó morto — fica para a próxima etapa.*

Abstract. *This work presents a distributed ant colony for the traveling salesman problem, run by symmetric Java nodes that exchange messages over gRPC. The pheromone matrix is replicated and may only change inside a critical section, whose coordination combines leader election with the Bully algorithm, heartbeat-based failure detection, Lamport logical clocks and Ricart-Agrawala mutual exclusion. In trials with four nodes and the berlin52 instance, the replicas converged to identical matrices and the cluster reached the 7542 tour, which is the known optimum. Leader failover took about 10 s on the local network profile, and the first round after the crash dropped from 15.6 s to 0.56 s after we fixed the broadcast. The central limitation — not telling a slow node from a dead one — is left for the next stage.*

Palavras-chave: computação distribuída; colônia de formigas; eleição de líder; exclusão mútua; relógio lógico

1. Introdução

O problema do caixeiro-viajante (TSP) pede o ciclo de menor comprimento que visite cada cidade uma vez e retorne à origem; por ser NP-difícil, instâncias práticas são atacadas por heurísticas. Usamos *berlin52*, da TSPLIB [5] (52 cidades, ótimo 7542), e uma instância aleatória de 60 cidades, ambas com distâncias euclidianas.

A colônia de formigas (ACO) [4] se distribui bem porque cada formiga é independente: lê a matriz de feromônio no início do passeio e devolve uma rota. O custo divide-se quase linearmente entre os nós, e o único estado compartilhado é a matriz $n \times n$ — um acoplamento pequeno e bem delimitado.

Construímos um *cluster* em Java 21 com gRPC. Todos executam o mesmo binário, e o papel de líder ou trabalhador vem da eleição, não da configuração. Cada nó roda um lote de formigas, entra na seção crítica para atualizar e propagar a matriz e informa sua melhor rota ao coordenador; a matriz é replicada em memória e o sistema roda em contêineres ou em máquinas da mesma rede local.

O enunciado exige dois requisitos; implementamos quatro: gRPC, relógio de Lamport, eleição por Bully e exclusão mútua por Ricart-Agrawala. As Seções 2 a 6 trazem arquitetura, requisitos, resultados, problemas e papéis.

2. Arquitetura do Sistema

Não existe nó mestre: todos executam o mesmo binário, e a configuração informa apenas `NODE_ID` e `PEERS`; o papel de líder vem da eleição e pode mudar durante a execução. O `RoundCoordinator`, ativo só no líder, incrementa a rodada, envia `StartRound` e aguarda os `RoundDone`, com prazo de 30 s; o `RoundWorker`, presente em todos os nós, executa as formigas, a seção crítica e a resposta a quem iniciou a rodada. O `LeaderMonitor` envia *heartbeat* a cada segundo e dispara eleição após cinco faltas.

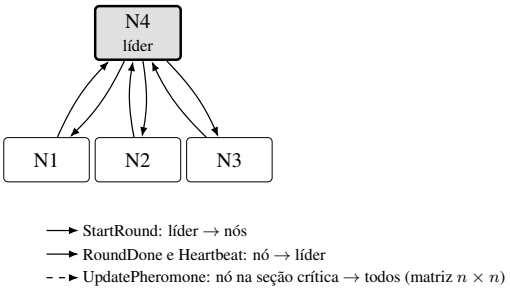


Figura 1. Entidades e mensagens. Todos os nós são trabalhadores; o líder (maior id vivo) coordena as rodadas, e a matriz de feromônio só muda dentro da seção crítica.

2.1. Mensagens

O serviço `NodeService` tem oito RPCs unários, sem *streaming*. A eleição leva o identificador do candidato; o pedido de seção crítica, o carimbo que o identifica, e a resposta devolve qual pedido está autorizando; a atualização leva a matriz e a melhor rota; os demais, a rodada ou o resultado local. Todo cabeçalho leva o remetente e o carimbo de Lamport, e o token do *cluster* viaja no *metadata* `x-cluster-token`, conferido por um interceptor antes do *handler*.

2.2. Uma rodada

Cada rodada usa semente distinta por nó e por rodada, de modo que a mesma configuração reproduz a execução. O nó roda `ITERATIONS_PER_ROUND` lotes de formigas sobre o mesmo *snapshot* e fica com o melhor; só então entra na seção crítica, onde evapora, deposita e transmite a matriz de forma síncrona. A parte cara roda fora dela e em paralelo, e o *broadcast* espera apenas os *peers* vivos.

3. Requisitos Implementados

3.1. RPC – gRPC

Usamos oito RPCs unários em um serviço, sem *streaming*. Toda chamada tem prazo configurável: 500 ms no *loopback* e 1500 ms em rede local, com prazos maiores para o *broadcast* e a espera do `COORDINATOR`. O serviço recusa remetente fora de `PEERS`. A decisão mais visível foi o limite de 64 MiB por mensagem: o padrão de 4 MiB estourava a matriz a partir de cerca de 725 cidades e o *peer* era marcado como morto.

3.2. Eleição de líder – Bully

A eleição [3] segue três passos: `ELECTION` só aos identificadores maiores; se ninguém responde OK, o nó se declara líder e envia `COORDINATOR`; se alguém responde, espera o `COORDINATOR` por 3 s e recomeça. Quem recebe `ELECTION` de identificador menor responde OK e inicia a própria eleição, e um `COORDINATOR` menor é contestado, o que restabelece o invariante quando um nó sobe depois. A detecção é separada: *heartbeat* de 1 s e cinco faltas, para não eleger por uma pausa de GC; o preço é cerca de 8,5 s de latência no perfil local, com aproximadamente 10 s até o anúncio do novo líder.

3.3. Relógio lógico – Lamport

O relógio lógico [1] incrementa em eventos locais e em envios e, ao receber uma mensagem, assume o maior valor entre o local e o recebido, somado de um. Ele ordena os pedidos de seção crítica — o carimbo é o primeiro componente do par de prioridade — e numera as linhas de log, o que permite reconstruir a ordem causal de uma execução. Não usamos relógio físico: para decidir quem espera, basta a ordem causal.

3.4. Exclusão mútua – Ricart-Agrawala

Cada nó está fora da seção crítica, querendo entrar ou dentro dela; ao pedir, congela um carimbo de Lamport e envia `REQUEST` a todos [2], com prioridade dada pelo par (carimbo, identificador). O `REPLY` carrega o carimbo do pedido que autoriza, então uma permissão atrasada não vale para um pedido futuro, e o custo é $2(N-1)$ mensagens por entrada. Para não travar, o pedido é reenviado a cada 500 ms e um *peer* que deixou de estar vivo não é aguardado — logo, a falha conta como permissão e dois nós podem entrar juntos se um apenas estiver lento. Assumimos esse risco; ele é discutido na Seção 5.

4. Resultados

As réplicas convergem ao final, e o `./tp check` confirma que nunca houve dois nós na seção crítica; o *cluster* atinge o ótimo conhecido em *berlin52*. A rodada leva cerca de 0,94 s, quase toda em CPU local, com a rede entrando uma vez por rodada. O *failover* leva cerca de 10 s até o anúncio do novo líder, dos quais 8,5 s são apenas o detector.

Medir o *failover* de ponta a ponta revelou um custo que o teste unitário não via: a primeira rodada após a queda levava 15,6 s, porque o *broadcast* aguardava o *peer* morto até o fim do prazo, inclusive quando ele era o coordenador; esperando só os vivos, caiu para 0,56 s.

Medida	Valor observado
Testes automatizados	43 em 12 classes, 0 falhas
Rodada (berlin52, 40 formigas/nó, I=1000)	≈0,94 s (mediana; 0,84–1,36 s)
Melhor rota em berlin52 (20–30 rodadas)	7542 (ótimo conhecido)
Failover do líder (4 nós, perfil LAN)	≈10 s até `Novo líder: N3`
1ª rodada após a queda	0,56 s (era 15,6 s antes do F14)
Execução longa / queda de seguidor	4000 rodadas sem timeout; 600 concluídas

Tabela 1. Medidas do sistema. Exceto a execução longa (auditoria de 20/10), os tempos vêm de uma máquina com 4 contêineres; I é `ITERATIONS_PER_ROUND`. Não medimos speedup de 1×N nós.

5. Desafios e Problemas Encontrados

O detector trabalha por *timeout* e não distingue nó lento de nó morto. Quando um `REQUEST` falha ou expira, o nó trata o silêncio como permissão: é o que impede o *cluster* de travar quando um *peer* morre, mas o mesmo caminho dispara quando ele apenas sofre uma pausa de GC. Reproduzimos o caso com dois nós reais, e ambos entraram na seção crítica. Não corrigimos: separar lento de morto exige quórum ou consenso, e privilegiamos a disponibilidade.

Já a queda de um *peer* entre o `REQUEST` e o `REPLY` deixava a aquisição presa, pois o nó aguarda $N - 1$ permissões; passamos a reenviar o pedido a cada 500 ms e a recontatar quem volta após a quarentena, o que também destravou a queda de seguidor, que custava 30 s por rodada. Configurações divergentes, por fim, geravam *split-brain* silencioso e agora são recusadas na subida, com código 2.

6. Papel de Cada Aluno

Pedro Henrique Vilaça Valverde ficou com a comunicação e a infraestrutura (prazos de rede local, *handshake* de token, TSPLIB, contêineres), com as Seções 2 e 4, o ensaio em duas máquinas e a montagem do texto. Marcus Prado Silva, com o Ricart-Agrawala e o Lamport (Seções 3.3, 3.4 e 5), a verificação de que nunca há dois nós na seção crítica e a documentação técnica. Gabriel Batista de Almeida ficou com o estudo do ACO e das rodadas (Seções 1 e 2.3) e com os números de *berlin52*; Eduardo Alves Salgado Lisboa Oliveira, com o Bully (Seção 3.2) e o *failover* cronometrado.

Referências

[1] LAMPORT, L. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, v. 21, n. 7, p. 558–565, 1978.

[2] RICART, G.; AGRAWALA, A. K. An optimal algorithm for mutual exclusion in computer networks. *Communications of the ACM*, v. 24, n. 1, p. 9–17, 1981.

[3] GARCIA-MOLINA, H. Elections in a distributed computing system. *IEEE Transactions on Computers*, v. C-31, n. 1, p. 48–59, 1982.

[4] DORIGO, M.; GAMBARDELLA, L. M. Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Trans. Evol. Comput.*, v. 1, n. 1, p. 53–66, 1997.

[5] REINELT, G. TSPLIB: a traveling salesman problem library. *ORSA J. Computing*, v. 3, n. 4, p. 376–384, 1991.